DSC Weather

Technical Guide

Weather Station Ingestion — Station Metadata & Readings Pipeline

Generated July 28, 2026

Workflow id: **dsc_weather**

1. Overview

The **dsc_weather** workflow ingests weather station observations from a configured EarthRanger subject group, tags each station with the conservancy boundary it falls within, and exports structured, analysis-ready station metadata and weather reading datasets. Unlike a per-survey pipeline, it performs a single, station-centric ingestion pass each run — there is no survey, activity-period, or connection fan-out; every task executes exactly once against the workflow's single Time Range.

Each run delivers:

- A **station metadata file** — one row per weather station, with its conservancy and coordinates
- **Per-station, per-year weather reading files** — precipitation, temperature, humidity, and wind speed observations
- Two **dashboard table widgets** exposing both outputs for review directly from the run's Overview dashboard

Output summary

Output type	Format	Description
station_metadata	Parquet	One row per weather station: Station ID, Conservancy, Latitude, Longitude
{station_id}_{year}	Parquet	One file per weather station per calendar year: Precipitation, Humidity, Temperature, Windspeed, fixtime_year, fixtime

Note: {station_id} and {year} are derived from the Station ID and fixtime_year columns respectively. Both output types are produced exactly once per workflow run — neither is prefixed with a survey or period name, since the workflow has no survey/period concept.

2. Dependencies

2.1 Python packages

The workflow declares six versioned packages from the Ecoscope prefix.dev channels:

Package	Version	Channel
ecoscope-platform	>=2.15.0, <2.16.0	ecoscope-workflows
ecoscope-workflows-ext-custom	0.1.0rc14.*	ecoscope-workflows-custom
ecoscope-workflows-ext-ste	0.0.0rc1.*	ecoscope-workflows-custom
ecoscope-workflows-ext-distance-sample-counts	1.0.2.*	ecoscope-workflows-custom
pydeck	0.9.2	conda-forge
opentelemetry-sdk	>=1.20.0, <2.0.0	conda-forge

Note: ecoscope-workflows-ext-ste supplies the spatial_join task used for conservancy tagging (Section 4.3); ecoscope-workflows-ext-distance-sample-counts supplies dedupe_by_column and split_weather_by_station_year (Sections 4.5 and 4.6). Neither pydeck nor a Google Earth Engine connection is actually exercised by any task in this workflow — pydeck is pinned for parity with the sibling dsc_analysis workflow this was split out of.

2.2 Weather EarthRanger connection

A single EarthRanger connection (**set_er_connection**, id: weather_er_client) plus a subject group name (**set_string_var**, id: weather_subject_group_name, default: "TAHMO Stations") configure the workflow's only data source.

3. Input Configuration

Parameter	Description
workflow_details	Human-readable name and optional description for this workflow run — used for dashboard registration
time_range	Required on all Ecoscope workflows. Controls which weather observations are fetched (Since/Until), and is used for timestamp display and UTC conversion
groupers	Set to an empty list by default (partial: groupers: []) — the workflow does not group its dashboard by any categorical field
weather_er_client	EarthRanger connection hosting the weather station subject group
weather_subject_group_name	EarthRanger subject group name containing the weather stations. RJSF title: "Weather Station Subject Group", default: "TAHMO Stations"

4. Data Ingestion Pipeline

4.1 Fetching station observations

get_subjectgroup_observations (id: fetch_weather_obs) retrieves observations for the configured weather_subject_group_name from weather_er_client, over the workflow's time_range (filter: clean, include_details: true, include_subjectsource_details: false, raise_on_empty: false — an empty result does not raise an error, it simply skips every downstream task via the any_is_empty_df / any_dependency_skipped skip policy, see Section 6.1).

4.2 Extracting weather variables

Four chained calls to **extract_value_from_json_column** pull numeric fields out of the extra__observation_details JSON column into flat float columns:

Task id	Output column	Source JSON field
extract_precipitation	Precipitation	precipitation
extract_temperature	Temperature	surface_air_temperature
extract_humidity	Humidity	relative_humidity
extract_wind_speed	Windspeed	wind_speed

4.3 Tagging stations with their conservancy

A one-time conservancy boundaries file (mara_conservancies.parquet) is downloaded from Dropbox via **fetch_and_persist_file** (id: fetch_conservancy_boundaries; overwrite_existing: false, retries: 3), loaded with **load_df** (id: load_conservancy_boundaries), and reprojected to EPSG:4326 (**reproject_gdf**, id: reproject_conservancy_boundaries). Each weather reading is then spatially joined to a conservancy boundary via **ecoscope_workflows_ext_ste.tasks.spatial_operations.spatial_join** (id: weather_conservancy_join; how: inner, predicate: intersects) — stations that don't fall within any boundary in the file are dropped from the weather output.

4.4 Identity, coordinates, and date/time decomposition

Station identity columns are renamed via **map_columns** (id: rename_weather_identity; extra__subject__name → Station ID, name → Conservancy). Latitude and longitude are extracted from the point geometry via **parse_df_point** (id: extract_weather_lat_long), and the fixtime timestamp is split into date, time, and year components via **decompose_datetime** (id: decompose_weather_fixtime; components: date, time, year), then renamed to Date and Time via a second **map_columns** call (id: rename_weather_datetime).

4.5 Station metadata output

Station ID, Conservancy, Latitude, and Longitude are selected via **select_columns** (id: select_station_metadata) and deduplicated to one row per station via **ecoscope_workflows_ext_distance_sample_counts.tasks.transformation.dedupe_by_column** (id: dedupe_station_metadata; column: Station ID). The result is persisted as **station_metadata.parquet** via **persist_df** (id: persist_station_metadata) — the only persist step in the workflow that is not fanned out via mapvalues.

4.6 Weather readings output

Station ID, Precipitation, Humidity, Temperature, Windspeed, fixtime_year, and fixtime are selected via **select_columns** (id: select_weather_readings) — deliberately omitting Conservancy/Latitude/Longitude, which live in station_metadata.parquet instead so the per-reading files stay slim. **ecoscope_workflows_ext_distance_sample_counts.tasks.transformation.split_weather_by_station_year** (id: weather_station_year_split) groups the readings by (Station ID, fixtime_year) and emits one [filename, DataFrame] pair per group, keyed **{station}_{year}** (e.g. TAHMO001_2026). Each pair is persisted via **persist_df** (id: persist_station_year_weather, filetype: parquet) using the standard filename/df mapvalues pattern.

> **Note:** To reconstruct a station's full weather record with location, join a {station_id}_{year}.parquet file back to station_metadata.parquet on Station ID.

4.7 Dashboard table widgets

Both outputs are additionally rendered as browsable table widgets and attached to the run's dashboard:

Widget	Source DataFrame	Task chain
Station Metadata	dedupe_station_metadata.return	draw_table (id: station_metadata_table_html) → persist_text (id: station_metadata_table_html_url) → create_table_widget_single_view (id: station_metadata_table_widget)
Station Year Weather Data	select_weather_readings.return	draw_table (id: syw_table_html) → persist_text (id: syw_table_html_url) → create_table_widget_single_view (id: syw_table_widget)

> **Note:** The Station Year Weather Data widget renders select_weather_readings' combined output — the DataFrame as it exists before weather_station_year_split fans it out into individual per-station-year files — so the widget shows every reading in one sortable/filterable table rather than one tiny table per station/year. Both draw_table steps share the same table_config (enable_sorting: true, enable_filtering: true, enable_download: false, hide_header: false).

5. Output Files

All outputs are written to **$ECOSCOPE_WORKFLOWS_RESULTS**.

File	Format	Description
station_metadata.parquet	Parquet	One row per weather station, produced once per run: Station ID, Conservancy, Latitude, Longitude
{station_id}_{year}.parquet	Parquet	One file per weather station per calendar year covered by the Time Range: Station ID, Precipitation, Humidity, Temperature, Windspeed, fixtime_year, fixtime

Note: If the configured weather station subject group returns no observations for the Time Range, fetch_weather_obs returns an empty DataFrame and the shared any_is_empty_df / any_dependency_skipped skip policy (Section 6.1) causes every downstream task — including both persist steps and both dashboard widgets — to be skipped gracefully rather than raising an error.

6. Workflow Execution Logic

6.1 Skip conditions

All tasks share a global default skip policy defined in **task-instance-defaults**:

- **any_is_empty_df** — skips the task if any upstream DataFrame dependency is empty
- **any_dependency_skipped** — skips the task if any upstream task was itself skipped

Because the workflow has a single linear chain from fetch_weather_obs through to the persist and widget steps, an empty observation fetch (or a conservancy join that drops every station) propagates a single skip signal all the way to station_metadata.parquet, every {station_id}_{year}.parquet file, and both dashboard widgets.

6.2 mapvalues fan-out for per-station-year persistence

The workflow is otherwise unfanned — every task runs exactly once — with one exception: **persist_station_year_weather** uses **mapvalues** (argnames: filename, df) over weather_station_year_split's list of [filename, DataFrame] pairs, so persist_df runs once per (station, year) group rather than once for the whole DataFrame.

6.3 Dashboard assembly

gather_dashboard (id: overall_dashboard) assembles the run's dashboard from workflow_details, time_range, groupers (empty), and both table widgets (station_metadata_table_widget, syw_table_widget). Since groupers is empty, both widgets use the *_single_view variant of their widget-creation task rather than a grouped/merged variant.

7. Software Versions

Package	Version pinned
ecoscope-platform	>=2.15.0, <2.16.0
ecoscope-workflows-ext-custom	0.1.0rc14.*
ecoscope-workflows-ext-ste	0.0.0rc1.*
ecoscope-workflows-ext-distance-sample-counts	1.0.2.*
pydeck	0.9.2
opentelemetry-sdk	>=1.20.0, <2.0.0

Note: All packages are resolved from the prefix.dev Ecoscope conda channels. The wildcard patch-version pin (.*) allows bug-fix releases to be picked up automatically while keeping minor and major versions locked.